

50.043 Database Systems

Project Lab 2 Report

Class: CI04

Names: Tan Li Hon (1008920) & Celeste Chan Shi Ting (1008994)

Design Decisions

Stateful left tuple for nested-loop join | [Join.java](#)

In `Join.java`, `fetchNext()` (line 115) uses a nested while loop which is simple and the least complex algorithm. The design decision made was to create a variable 'left' to store the current left tuple. Since `fetchNext()` returns 1 tuple at a time, we need to remember which left tuple is being processed across multiple calls. If this is not implemented, it might cause 1 left tuple that matches multiple right tuples to produce only 1 result instead of all possible results.

Single-pass aggregate state per group | [IntegerAggregator.java](#)

`IntegerAggregator` uses a `LinkedHashMap` to store the aggregate state per group. Each group will map into an array of integers that stores the count, sum, min and max. All 4 values will be computed in `mergeTupleIntoGroup()` (line 56) regardless of the operation being requested. Thus, this will help avoid tuples needing to re-scan if `iterator()` is being called. We chose to implement a `LinkedHashMap` instead of a normal `HashMap` as it can preserve the insertion order of groups, hence ensuring the results returned are in the same order as they were processed.

LRU Page Eviction | [BufferPool.java](#)

When the buffer pool reaches `numPages`, a page must be evicted before a new one is loaded. We chose LRU over Clock. A `LinkedList<Pages>` tracks access recency, where the front holds the least recently used page and the back holds the most recent. Every `getPage()` calls a `touch()` helper function that removes the page id and reappends it to the back so the recency ordering stays current on both the cache hits and fresh loads. `evictPage()` removes the front entry. LRU was chosen as the `LinkedList` gives a direct, readable ordering of recency without the extra reference-bit bookkeeping that Clock requires, at the tradeoff of $O(n)$ removal from the middle of the list on each touch.

Writes routed through the HeapFile | [BufferPool.java](#)

`BufferPool.insertTuple()` and `deleteTuple()` do not modify pages directly. They locate the table's `DbFile` via the Catalog and delegate to `HeapFile.insertTuple` or `deleteTuple`, then mark every returned page dirty and place it back into cache. This extra level of indirection keeps the buffer pool agnostic to file type so that other files like indices can be supported in future labs.

Flush and evict kept separate | [BufferPool.java](#)

`FlushPage()` writes a dirty page to disk and clears its dirty bit but leaves the page in cache. `EvictPage()` is the only method that removes a page from the pool, calling `flushPage()` first if the victim is dirty. This separation ensures that `flushAllPages()` and `flushPage()` never shrink the pool, which is required for `ScanTest.cacheTest` to pass.

Non-Trivial Parts

Preserving join state across fetchNext calls | [Join.java](#)

In `Join.java`, `fetchNext()` (line 115) is a non-trivial part. Since it returns 1 tuple at a time, the join state must be preserved between calls. Thus, a left variable that stores the current left tuples in the field must be implemented. When `fetchNext()` is being called again, it resumes the scanning from where it stopped at instead of restarting again from the beginning. Once the right relation finishes scanning for a given left tuple, `child2.rewind()` is being called so that it can be reset and left is set to null. This will allow each left tuple to be compared with every right tuple and produce the matching pairs. When a match is found, the two tuples will be merged to a single tuple by copying the fields from the left tuple first then the right tuple using a merged `TupleDesc` from both children.

Running aggregate with sentinel min/max initialization | [IntegerAggregator.java](#)

In `IntegerAggregator.java`, `mergeTupleIntoGroup()` (line 56) is the non-trivial part. It maintains running aggregate across multiple tuple insertions. A linked hash map was used to store an array of four integer values (count, sum, min, max). On every call, the four values will be updated simultaneously. When initializing min and max, we did not choose to put 0 as the default value as it will cause inaccurate results. Instead, we used `Integer.MAX_VALUE` for min and `Integer.MIN_VALUE` for max. For `NO_GROUPING`, null is used as the key as linked hash maps allow null as a key. Thus, it will allow the code to handle both grouped and ungrouped aggregates.

Insert with free-slot search and page append | [HeapFile.java](#)

`InsertTuple()` scans existing pages through `BufferPool.getPage(0` with `READ WRITE` permission, inserting into the first page that reports a free slot. If no page has an empty slot, a new empty `HeapPage` is created, written to disk via `writePage()` to physically grow the file, then fetched back through the buffer pool before the tuple is inserted. Fetching through the buffer pool instead of holding the local page object ensures that the cached version stays authoritative and the `RecordId` is updated correctly.

Header bitmap mutation | [HeapPage.java](#)

`InsertTuple()` and `deleteTuple()` modify the header bitmap using `markSlotUsed()`. The bit order in `markSlotUsed()` mirrors `isSlotUsed()` from Lab 1 (`i/8` locates the bit, `i%8` locates the bit), so reads and writes stay consistent. Insert finds the first free slot, flips its bit on, stores the tuple and updates its `RecordId`. Delete validates that the tuple belongs to the page and its slot is used before flipping the bit off and clearing the slot. A dirty flag and the dirtying transaction id are tracked so `isDirty()` can report which transaction last modified the page.

Single-return count from Insert/Delete | [Inert.java](#), [Delete.java](#)

Both operations drain their child, calling the matching `BufferPool` method per tuple, and return a single one-filed tuple holding the count of affected records. A called flag guards `fetchNext()` so that the count tuple is returned only once, and a second call returns null. This matches the query-plan contract where an insert or delete produces exactly one summary row.

API Changes

No changes to API.

Missing or Incomplete Elements

1. `StringAggregator` only supports COUNT. SUM, AVG, MIN and MAX are not implemented.
2. Aggregates over multiple fields or grouping by multiple fields are not implemented.
3. Transactions and locking are not implemented. `EvictPage()` may evict dirty pages, which is acceptable for Lab2.

